

02_m3_m4

M3 — Sessionization & Funnel KEYSTONE [Wk1-MVP]

This is the highest-stakes milestone in the build. `int_ga4_sessionized` and `int_ga4_funnel_steps` reconstruct the two entities GA4 never ships as rows — the **session** and the **funnel** — and they **fail silently**: a wrong MD5 concat order still yields a valid-looking key, a non-monotonic flag still returns a number, a `traffic_source` first-touch leak still returns rows. Nothing throws; the numbers are just quietly wrong, and every mart, metric, and agent diagnosis inherits the corruption. The non-negotiable operating rule for M3 is therefore **TEST-FIRST**: the golden-value unit tests + the two singular tests are written and watched fail *before* the model exists. The code already lives in the docs — Claude Code copies/adapts it, it does not invent it.

Context to load

- **Root CLAUDE.md** (auto-loads). It pins the four canonical names you will check by eye in every diff: `session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))`; `reached_*` max-downstream monotonic (`did_*` retired); `engaged_session = session_engaged = '1'` OR `engagement_time_msec ≥ 10000`; the `traffic_source` first-touch FALLBACK.
- **@docs/DBT_GUIDE.md §4** — the full copy-usable SQL for both keystones (§4.1 `int_ga4_sessionized`, §4.2 `int_ga4_funnel_steps`), the intermediate `schema.yml` (§4.3), and the keystone test budget (§4.4).
- **@docs/DBT_GUIDE.md §6.3 + §6.5** — the singular tests (`assert_funnel_monotonicity`, `assert_session_conversion_rate_bounds`) and the three golden-value `unit_tests` (`test_sessionize_key_construction`, `test_reached_flags_are_max_downstream`, `test_decomposition_inputs_additive`).
- **@docs/DATA_MODEL.md §5** — the session model as spec: §5.1 `session_key` + cookie-grain honesty, §5.3 the `traffic_source` FALLBACK, §5.4 engaged/new-user, §5.5 `int_ga4_sessionized` grain/PK/columns, §5.6 the `reached_*` monotonic chain.
- The **upstream models** Claude must reference for real columns: `@models/staging/stg_ga4_events.sql` (it provides `session_key`, `event_source / event_medium`, `first_touch_source / first_touch_medium`, `gclid`, `session_engaged`, `engagement_time_msec`, `ga_session_number`, `transaction_id`, `purchase_revenue_in_usd`) and `@macros/sessionize.sql` + `@macros/channel_group.sql`.

Run M3 in a **dedicated session after /clear** . Plan mode first (D9): ask for the file list + the test-first order, check it against §4 before any code is written.

Files to provide

Attach these and nothing else (a tight context is the anti-drift lever and the Pro-budget lever):

Attach	Why
@docs/DBT_GUIDE.md §4.1, §4.2, §4.3	source-of-truth SQL + intermediate schema tests (D2)
@docs/DBT_GUIDE.md §6.3, §6.5	the singular + golden-value tests to write FIRST
@docs/DATA_MODEL.md §5.1, §5.3, §5.5, §5.6	grain/PK/column spec + the FALLBACK + the monotonic chain
@models/staging/stg_ga4__events.sql	the real upstream columns (D3 — so it can't invent a column name)
@macros/sessionize.sql , @macros/channel_group.sql	the single sources of truth it must call, not re-derive

Prompts to use

Prompt 1 — write the golden tests FIRST (this is the gate, not the model):

```
[ROLE] You are implementing Helios milestone M3. Write ONLY the keystone tests, before any model SQL
[SOURCE OF TRUTH] Copy/adapt the three unit_tests in @docs/DBT_GUIDE.md §6.5 into
models/intermediate/_int__unit_tests.yml, and the two singular tests in §6.3 into
tests/assert_funnel_monotonicity.sql and tests/assert_session_conversion_rate_bounds.sql.
Follow CLAUDE.md conventions (auto-loaded).
[TASK] Reproduce: test_sessionize_key_construction (u1+100 collapses 2 events to 1 session,
channel_group='Paid Search' from gclid+cpc, engaged), test_reached_flags_are_max_downstream
(a purchase-only session sets EVERY reached_* true; an add_to_cart-only session sets
reached_view_item+reached_add_to_cart true and downstream false), and the monotonicity
singular SELECT (returns offending sessions where reached_purchase > reached_add_payment_info > ..
[CONSTRAINTS] Use ONLY canonical names: session_key, reached_view_item/add_to_cart/begin_checkout/
add_shipping_info/add_payment_info/purchase, session_revenue, channel_group. Do NOT use did_*.
Do NOT invent a column. If a name is not in §6.5/§6.3 or stg_ga4__events, STOP and ASK.
[TEST-FIRST] Then run `dbt parse` to confirm the YAML + SQL parse. Do NOT write the models yet.
[OUTPUT] Cite the exact doc subsection (§6.5 / §6.3) used for each test.
```

Prompt 2 — implement `int_ga4__sessionized` to pass the sessionization unit test:

```
[ROLE] You are implementing Helios milestone M3, file models/intermediate/int_ga4__sessionized.sql.
[SOURCE OF TRUTH] Copy/adapt the SQL in @docs/DBT_GUIDE.md §4.1; cross-check grain/PK/columns
```

against @docs/DATA_MODEL.md §5.5 and the FALLBACK rule in §5.3. Follow CLAUDE.md (auto-loaded).

[UPSTREAM] Reference only @models/staging/stg_ga4__events.sql for columns and call {{ sessionize() }} (@macros/sessionize.sql) and {{ channel_group_case(...) }} (@macros/channel_group_case.sql).

[TASK] One row per session (group by user_pseudo_id, ga_session_id; drop null ga_session_id). Derive: landing_page = page_location of the EARLIEST event (array_agg(... order by event_timestamp limit 1)); session_source/medium = COALESCE(session-scoped event_source/medium, traffic_source first_touch_*) – the FALLBACK; channel_group via channel_group_case() on the RESOLVED source/medium + gclid; engaged_session = session_engaged='1' OR engagement_time_msec ≥ 10000; is_new_user = (ga_session_number = 1). Materialize ephemeral.

[CONSTRAINTS] session_key comes ONLY from sessionize(); channel_group ONLY from channel_group_case() on the resolved (post-fallback) source/medium – NEVER from the raw traffic_source struct. Use ONLY columns present in stg_ga4__events. If one is missing, STOP and ASK – do not guess.

[TEST-FIRST] Make test_sessionize_key_construction pass: run `dbt build --select int\_ga4\_\_sessionized` and show me the test output.

[OUTPUT] Cite the §4.1 / §5.3 / §5.5 line behind each derived column.

Prompt 3 — implement `int_ga4__funnel_steps` to pass the monotonicity tests:

[ROLE] You are implementing Helios milestone M3, file models/intermediate/int_ga4__funnel_steps.sql.

[SOURCE OF TRUTH] Copy/adapt @docs/DBT_GUIDE.md §4.2; cross-check the monotonic chain in @docs/DATA_MODEL.md §5.6. Follow CLAUDE.md (auto-loaded).

[UPSTREAM] Reference only @models/staging/stg_ga4__events.sql.

[TASK] One row per session. Each reached_X = LOGICAL_OR(event_name IN {X and ALL downstream stages}) (max-downstream). session_revenue = deduped purchase revenue: one value per (session_key, transaction_id) via MAX over the partition, then SUM across distinct transactions, COALESCE 0. Materialize ephemeral.

[CONSTRAINTS] reached_* names are canonical and max-downstream – did_* is FORBIDDEN. The stage sets must nest exactly: reached_view_item's IN-list contains all six stages, reached_purchase's only 'purchase'. Do NOT invent stage names. If unsure, STOP and ASK.

[TEST-FIRST] Make test_reached_flags_are_max_downstream pass first: `dbt build --select int\_ga4\_\_funnel\_steps`. Then run `dbt build --select int\_ga4\_\_funnel\_steps, test\_type:unit` and show output.

[OUTPUT] Cite §4.2 / §5.6 for the flag logic and the dedup logic.

Preventing hallucinations

- **Golden tests are the firewall (D6).** The two keystones cannot be caught by `not_null` / `unique` alone — those test shape, not correctness. The `test_reached_flags_are_max_downstream` unit test runs on synthetic input every build in <1s and is what catches a non-monotonic flag *before* any real data flows; demand it green before accepting the model. `dbt build` (not `run` then `test`) interleaves test execution in DAG order so a failed keystone test aborts before the poison reaches `fct_funnel`.
- **Pin the four exact expressions in the prompt and re-verify each by eye.** The `session_key` concat order (`user_pseudo_id` then `'-'` then `CAST(ga_session_id AS STRING)`), the FALLBACK direction (`COALESCE(event_source, first_touch_source)` — session first, first-touch second, never the reverse), the `reached_*` IN-list nesting, and

`engaged_session` 's $\geq$ (not $>$). A reversed FALLBACK is a Simpson's-paradox-grade silent bug.

- **Forbid `did_*` explicitly (D5).** `grep -rn "did_" models/intermediate/` must return nothing except the deliberately-retained `did_session_start` denominator anchor if §4.2 uses it; any other `did_` flag means Claude reintroduced the retired naming.
- **`channel_group` only via the macro, on the resolved source/medium (D7).** If Claude inlines a `CASE WHEN ... THEN 'Paid Search'`, reject it — channel logic lives only in `channel_group_case()`; an 11th group or a hand-rolled CASE is a hard error.
- **Attach `stg_ga4__events.sql` so column names are physical (D3).**
`event_source` / `event_medium` (session-scoped) vs
`first_touch_source` / `first_touch_medium` (user first-touch) are easy to swap — having the real file open means Claude reads the names rather than guessing.

Reviewing generated code

Apply the rubric, specialized to the keystones:

- **R3 (ran the test) — the decisive check.** Did Claude show `test_sessionize_key_construction` and `test_reached_flags_are_max_downstream` *passing*? No green unit test $\Rightarrow$ do not accept. Then `dbt test --select int_ga4__sessionized int_ga4__funnel_steps`.
- **R5 (read the diff by eye) —** read these three lines against the spec, every time: (1) the `session_key` expression matches `TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))` character-for-character (it should come from `sessionize()`, so confirm the call, not a re-typed expression); (2) the FALLBACK is `COALESCE(<session-scoped>, <first-touch>)` and not reversed; (3) the `reached_*` roll-forward — `reached_view_item` LOGICAL_OR's the set of all six downstream stages, each subsequent flag drops the leftmost stage, `reached_purchase` is 'purchase' only.
- **R2/R6 (compile + dry_run) —** `dbt compile --select int_ga4__sessionized int_ga4__funnel_steps` (ephemeral models don't materialize standalone, so compile is how you confirm refs + macros resolve). Then `dbt build --select +fct_funnel --full-refresh` and confirm `assert_funnel_monotonicity` returns 0 rows. A BigQuery `dry_run` on the compiled SQL catches any hallucinated column not in `stg_ga4__events`.
- **R4 (canonical names) —** `grep -n "channel_group" models/intermediate/*.sql` resolves to the macro call; no inline CASE. `grep "did_" clean (modulo did_session_start)`. The `accepted_values` test on `channel_group` lists exactly the 10 groups.

Then `/clear` **before M4** — the intermediates are done; carrying their context into M4 only adds drift and burns budget.

M4 — Marts [Wk1-MVP]

M4 assembles the **consumption layer** — the only models the semantic layer (and therefore the agents) may read. The keystones did the hard logic; marts *assemble, they do not re-derive*. The job is wide, conformed, grain-explicit facts: `fct_sessions` and `fct_funnel` (session grain, incremental), `fct_daily_funnel` (additive day × dims, incremental), `fct_orders` / `fct_order_items` (finance, table), `dim_channels` / `dim_users` / `dim_items` / `dim_date` (table), and `fct_cohorts` / `fct_funnel_by_dim` (growth, table). The two machine-checks that define "done" are **revenue_reconciles to the cent** and **dim_channels accepted_values = exactly 10**; everything else is uniqueness, referential integrity, and a clean `dry_run`.

Context to load

- **Root CLAUDE.md** (auto-loads) — the WIDE-marts rule, the five base grains (`fct_funnel` , `fct_sessions` , `fct_orders` , `fct_order_items` , `fct_cohorts`), revenue = `*_in_usd` , rates computed `SUM(num)/SUM(den)` *in the semantic layer* (never stored in a mart).
- **@docs/DBT_GUIDE.md §5** — the full mart catalog (§5.1), why WIDE (§5.2), the incremental config (§5.3), per-mart grain/columns (§5.4–5.6), and the copy-usable SQL: §5.8 `fct_funnel` , §5.9 `fct_daily_funnel` , §5.10 `fct_orders` , §5.11 `dim_channels` , §5.12 `fct_cohorts` , plus the schema.yml tests (§5.13).
- **@docs/DBT_GUIDE.md §6.4** — the custom generic test `test_revenue_reconciles` (mart total == raw `purchase_revenue_in_usd` , tolerance 0).
- **@docs/DATA_MODEL.md §3–8** — the grains as spec: §5.7 `fct_sessions` , §5.8 `fct_funnel` (1:1 with `fct_sessions`), §5.9 `fct_daily_funnel` (additive counts, rates NOT stored), the order/item grains, and the cohort grain.
- The **upstream models** for each mart: `@models/intermediate/int_ga4__sessionized.sql` + `@models/intermediate/int_ga4__funnel_steps.sql` (feed `fct_funnel` / `fct_sessions`), `@models/marts/core/fct_funnel.sql` (feeds `fct_daily_funnel` — NOT the intermediate), `@models/staging/stg_ga4__events.sql` (feeds `fct_orders`), `@macros/channel_group.sql` (the 10-group source of truth `dim_channels` enumerates).

Run M4 in a **fresh session**. Plan mode first: confirm the build order `fct_sessions` → `fct_funnel` → `fct_daily_funnel` , `dim_*` independently, `fct_orders/items` after `fct_sessions` , `fct_cohorts` after `fct_sessions` .

Files to provide

Attach	Why
@docs/DBT_GUIDE.md §5.8–5.13	the copy-usable mart SQL + schema.yml tests (D2)
@docs/DBT_GUIDE.md §6.4	the test_revenue_reconciles generic test
@docs/DATA_MODEL.md §5.7–5.9	grain/PK/FK/column spec for each fact
@models/intermediate/int_ga4__sessionized.sql , @models/intermediate/int_ga4__funnel_steps.sql	the real upstream columns fct_funnel / fct_sessions ref (D3)
@models/marts/core/fct_funnel.sql	the upstream of fct_daily_funnel (it aggregates this, NOT the intermediate)
@macros/channel_group.sql	the 10 canonical groups dim_channels must enumerate exactly

Prompts to use

Prompt 1 — the conformed dim + the session facts, with their tests:

```
[ROLE] You are implementing Helios milestone M4, files models/marts/core/dim_channels.sql,
fct_sessions.sql, fct_funnel.sql and models/marts/core/core__schema.yml.
[SOURCE OF TRUTH] Copy/adapt @docs/DBT_GUIDE.md §5.11 (dim_channels), §5.8 (fct_funnel),
and §5.4 (fct_sessions columns); tests from §5.13 (core__schema.yml). Cross-check grain/PK/FK
against @docs/DATA_MODEL.md §5.7–5.8. Follow CLAUDE.md (auto-loaded).
[UPSTREAM] Reference only @models/intermediate/int_ga4__sessionized.sql and
@models/intermediate/int_ga4__funnel_steps.sql; the 10 groups in dim_channels are the exact
strings in @macros/channel_group.sql.
[TASK] dim_channels = exactly 10 rows (Direct, Organic Search, Paid Search, Display, Paid Social,
Organic Social, Email, Affiliates, Referral, Other) with is_paid/is_organic/channel_group_order
and channel_key = to_hex(md5(channel_group)). fct_sessions + fct_funnel: session grain (PK
session_key), WIDE denormalized dims, fct_funnel joins the two intermediates 1:1 USING(session_key
and carries the reached_* flags + session_revenue. Use the incremental insert_overwrite config
from §5.3 (partition_by event_date, cluster_by [device_category, channel_group],
require_partition_filter=true).
[CONSTRAINTS] WIDE marts – no rates stored (session_conversion_rate etc. live ONLY in the semantic
layer). channel_group strings must match channel_group_case() exactly – exactly 10, no 11th group,
no "Paid Other". Use ONLY columns present in the intermediates. If one is missing, STOP and ASK.
[TEST-FIRST] Add the schema.yml tests (unique/not_null PK, channel_group accepted_values = the 10,
relationships fct_funnel.channel_group → dim_channels), then
```

```
`dbt build --select dim_channels fct_sessions fct_funnel --full-refresh` and show the test output.  
[OUTPUT] Cite §5.8/§5.11/§5.13 per block.
```

Prompt 2 — the additive daily rollup + the finance fact with the reconcile test:

```
[ROLE] You are implementing Helios milestone M4, files models/marts/core/fct_daily_funnel.sql,  
models/marts/finance/fct_orders.sql, finance__schema.yml, and the generic test  
tests/generic/test_revenue_reconciles.sql.  
[SOURCE OF TRUTH] Copy/adapt @docs/DBT_GUIDE.md §5.9 (fct_daily_funnel), §5.10 (fct_orders),  
§6.4 (test_revenue_reconciles), tests from §5.13. Cross-check @docs/DATA_MODEL.md §5.9.  
Follow CLAUDE.md (auto-loaded).  
[UPSTREAM] fct_daily_funnel aggregates @models/marts/core/fct_funnel.sql (which carries  
session_revenue) - NOT int_ga4__funnel_steps. fct_orders reads @models/staging/stg_ga4__events.sql  
(purchase events) + the wide dims from fct_sessions.  
[TASK] fct_daily_funnel: grain event_date × [channel_group, device_category, country, is_new_user],  
daily_funnel_key = md5 of grain, additive counts only (sessions, view_item_sessions ...  
purchasing_sessions, transactions, revenue=SUM(session_revenue)) - NO rates. fct_orders: one  
deduped row per transaction_id (PK order_key), any_value() per txn, net_revenue = gross - refund,  
wide channel/device/date dims. Materialize fct_orders as table; fct_daily_funnel incremental.  
[CONSTRAINTS] fct_daily_funnel MUST ref fct_funnel (revenue must survive). Aggregate only *_in_usd  
revenue. Do NOT store any rate. Use ONLY canonical column names; if absent, STOP and ASK.  
[TEST-FIRST] Wire test_revenue_reconciles on fct_orders.gross_revenue (tolerance 0) and on  
fct_funnel.session_revenue, then `dbt build --select fct_funnel fct_daily_funnel fct_orders  
--full-refresh` and show that revenue_reconciles passes and purchasing_sessions ≤ sessions holds.  
[OUTPUT] Cite §5.9/§5.10/§6.4 per block.
```

Preventing hallucinations

- **fct_daily_funnel MUST aggregate fct_funnel, not int_ga4__funnel_steps (D2/D3).** This is the single most common M4 drift. fct_funnel carries session_revenue; the intermediate funnel-steps model does too, but the spec (§5.9, CLAUDE.md keystone #5) pins fct_funnel as the source so the dollar-at-risk labels the eval needs are preserved. Verify the ref(): `grep -n "ref(" models/marts/core/fct_daily_funnel.sql` must show `ref('fct_funnel')`.
- **dim_channels is exactly 10 rows, enumerated to match the macro (D7).** Reject any 11th group, "Paid Other", or a string that disagrees with channel_group_case() in @macros/channel_group.sql. The accepted_values test in §5.13 lists the 10 — they must match the macro's emitted strings character-for-character.
- **WIDE, no stored rates (D5).** If Claude adds session_conversion_rate, aov, or any ratio column to a mart, reject it — rates live only in the semantic layer (M5), computed SUM(num)/SUM(den) after grouping. Marts store additive primitives.
- **Let the reconcile + dry_run catch invented columns (D6).** test_revenue_reconciles (§6.4) sums the mart's revenue against raw purchase_revenue_in_usd and fails on >tolerance drift — a hallucinated/double-counted revenue column surfaces here. A

BigQuery dry_run on each compiled mart fails on any column not present upstream and prints the byte estimate (keep it inside the 5 GiB run budget).

- **Grain is explicit, one entity per mart.** `fct_funnel` = session, `fct_orders` = transaction, `fct_order_items` = order line. If a generated mart mixes grains (e.g. order columns on the session fact), reject it.

Reviewing generated code

- **R3/R6 — reconcile to the cent, then dry_run.** The acceptance gate is `dbt build --select fct_orders fct_funnel --full-refresh` showing `revenue_reconciles` green (mart `gross_revenue` total == raw to the cent, tolerance 0 on `fct_orders`; 0.5% on `session_revenue`). Then dry_run each mart for hallucinated columns + cost.
- **R4 — the 10-channel closed set.** `dbt test --select dim_channels` must pass `accepted_values` = exactly the 10 groups, and `unique + not_null` on `channel_key / channel_group` (10 rows, no dupes, no 11th). Confirm the `relationships` test from `fct_funnel.channel_group / fct_orders.channel_group` → `dim_channels` passes (referential integrity of the conformed dim).
- **R5 — read the diff by eye for two things.** (1) `fct_daily_funnel` refs `fct_funnel` (not the intermediate) and selects `SUM(session_revenue) as revenue` — revenue must survive the rollup. (2) The session facts are WIDE (dims denormalized onto the row) and store no rate column. Confirm the incremental config block (`insert_overwrite`, `partition_by event_date`, `cluster_by [device_category, channel_group]`, `require_partition_filter=true`) matches §5.3.
- **R2 — monotonicity at the rollup grain.** `fct_daily_funnel` carries the `dbt_utils.expression_is_true chain ( purchasing_sessions ≤ ... ≤ view_item_sessions ≤ sessions );` confirm it is present and green — it is the additive-grain twin of the keystone monotonicity test.

Then `/clear` before M5 (semantic layer) — the marts are now the contract; M5 reads `semantic_layer.yaml` + the marts, and a clean context keeps the registry validation focused.